Python Logic Building Workbook

Designed for Beginners to Intermediate Students

Instructions: Solve the algorithm first, then write Python code. Do not immediately jump into coding.

Exercise 1: Even or Odd

Difficulty: ■

Problem Statement
Read an integer and determine whether it is even or odd.

Sample Input: 15

Sample Output: Odd

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 2: Largest of Three

Difficulty: ■

Problem Statement
Find the largest of three numbers without using max().

Sample Input: 10 25 18

Sample Output: 25

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 3: Leap Year

Difficulty: ■

Problem Statement
Check if a year is a leap year.

Sample Input: 2024

Sample Output: Leap Year

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 4: Factorial

Difficulty: ■

Problem Statement
Find factorial of a positive integer.

Sample Input: 5

Sample Output: 120

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

Python Code (Student Space)

Exercise 5: Reverse Number

Difficulty: ■■

Problem Statement
Reverse the digits of a number.

Sample Input: 12345

Sample Output: 54321

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 6: Palindrome Number

Difficulty: ■■

Problem Statement
Check whether a number is palindrome.

Sample Input: 121

Sample Output: Palindrome

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 7: Prime Number

Difficulty: ■■

Problem Statement
Check whether a number is prime.

Sample Input: 29

Sample Output: Prime

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 8: Fibonacci

Difficulty: ■■

Problem Statement
Print first N Fibonacci numbers.

Sample Input: 8

Sample Output: 0 1 1 2 3 5 8 13

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 9: Sum of Digits

Difficulty: ■■

Problem Statement
Find sum of digits.

Sample Input: 4567

Sample Output: 22

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 10: Count Digits

Difficulty: ■■■

Problem Statement
Count digits in a number.

Sample Input: 987654

Sample Output: 6

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 11: Multiplication Table

Difficulty: ■■■

Problem Statement
Print table of a number up to 10.

Sample Input: 7

Sample Output: 7 x 1 = 7 ...

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 12: Count Vowels

Difficulty: ■■■

Problem Statement
Count vowels in a string.

Sample Input: education

Sample Output: 5

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 13: Reverse String

Difficulty: ■■■

Problem Statement
Reverse a string without slicing.

Sample Input: python

Sample Output: nohtyp

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 14: Character Frequency

Difficulty: ■■■

Problem Statement
Count frequency of each character.

Sample Input: banana

Sample Output: b:1 a:3 n:2

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 15: Second Largest

Difficulty: ■■■■

Problem Statement
Find second largest element in a list.

Sample Input: 12 45 67 89 34

Sample Output: 67

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 16: Remove Duplicates

Difficulty: ■■■■

Problem Statement
Remove duplicate values from a list.

Sample Input: 1 2 2 3 4 4

Sample Output: 1 2 3 4

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 17: ATM Menu

Difficulty: ■■■■

Problem Statement
Create a menu-driven ATM using loops and functions.

Sample Input: -

Sample Output: -

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 18: Guess Number

Difficulty: ■■■■

Problem Statement
Build a guess-the-number game.

Sample Input: -

Sample Output: -

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 19: Student Result

Difficulty: ■■■■

Problem Statement
Store marks, calculate total, average and grade.

Sample Input: -

Sample Output: -

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Exercise 20: Word Frequency

Difficulty: ■■■■■

Problem Statement
Count occurrences of each word in a sentence.

Sample Input: Python is easy and Python is powerful

Sample Output: Python:2

Hint: Break the problem into smaller steps. Think about variables, conditions, or loops before coding.

Algorithm (Student Space)

_____

_____

_____

_____

_____

_____

Python Code (Student Space)

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Solution Writing Template

For every problem:
1. Identify Inputs
2. Identify Outputs
3. Write Algorithm/Pseudocode
4. Implement in Python
5. Test with normal and edge cases
6. Discuss Time and Space Complexity